分享 · Chester

AI创作：什么是最重要的?

AI CREATION · SHENZHEN

AI 创作，什么是最重要的？

一个信息化项目经理，用 7 天做出第一款 AI 产品。

01

为什么说，AI 创作始于一个真实的提问？

02

同样用 AI，为什么有人问出来的东西完全不一样？

03

一个普通人，如何用一套协作流程，把想法变成能用的产品？

工具会换，提问的能力不会。



Chester Lee

INTJ · 现居深圳 · 信息化 PM × 独立开发者（AI 产品）

作品 | 微信悦读（浏览器插件）· 悦读且住（微信小程序）· H5 / 官网

AI 创作分享会 · 线下 · 深圳 · 约两小时

带上你最近一次「用 AI 没问出来」的问题

AI 是从哪一年开始的

1943

麦卡洛克与皮茨用数学描述一个神经元，第一个神经元模型。

1983

沃尔夫勒姆第一次写神经网络，「它并没有做任何有趣的事情」。

2019

GPT-2，书里用来演示「一次只加一个词」的模型。

2022

ChatGPT 发布。40 年后，计算机快了百万倍。

书里给的答案：神经网络诞生在 1940 年代，不是 2022 年。

它始于一个关于大脑的问题

ChatGPT 基于神经网络的概念，神经网络最初是在 1940 年代作为大脑运作的理想化而发明的。我自己在 1983 年首次编写了一个神经网络，但它并没有做任何有趣的事情。但是，40 年后，随着计算机的速度快了百万倍，互联网上有数十亿页的文本，并且进行了一系列工程创新，情况已经完全不同了。出人意料的是，一个比我 1983 年使用的神经网络大十亿倍的神经网络能够生成有意义的人类语言，这曾被认为是独特的人类特征。

—— 斯蒂芬·沃尔夫勒姆《这就是 ChatGPT》前言

AI 的根，长在物理里

01

约翰·霍普菲尔德

物理学家。1982 年提出霍普菲尔德网络：借物理里的「能量」解释联想记忆——给一半，能想起全部。

02

杰弗里·辛顿

计算机科学家。1970 年代开始研究神经网络，1985 年做出玻尔兹曼机，2006 年带火深度学习，2012 年 AlexNet 夺冠后神经网络被大规模应用——今天的大模型仍走这条路。

03

为什么是物理学奖

2024 年诺贝尔物理学奖授予两人，表彰「为利用人工神经网络实现机器学习奠定基础性发现与发明」。

从 1940 年代起的这条线，走了八十年，走到了斯德哥尔摩。

PART 01

从一个问题开始

今天不讲结论，先讲问题

01

02 · 先问一句

AI 创作，最重要的是什么？

先别急着回答，我们把它拆开看。

今天会留几个问题给大家，欢迎随时打断我。

PART 02

一切源于一个真实的提问

02

问题不同，答案的高度就不同

三类回答

如果问对土地改革过程感兴趣的青年学者他们研究的问题是什么，他们的回答一般会呈现三类形式，例如：

(1)「我想知道土地改革过程的发生和发展过程。」

(2)「我想知道土地改革过程中『情感动员』（或任何一个其他的视角）是怎么进行的。」

(3)「我发觉土地改革过程在各地呈现出许多不同，例如福建土地改革的斗争对象往往是宗祠而不是地主，『小土地出租者』这一成分划分在江南一带出现得较多，在有些地方土地改革过程首先发动起来的是一些社会边缘群体……我想知道为什么土地改革过程在各地区呈现出这些差异。」

以上三类回答中，前两类回答中的发问都属于过程性发问，第三类发问则属于差异性发问。

——《质性社会学研究的差异性发问和发问艺术》

同一个题目，三种问法

01	**过程性发问**	「我想知道土地改革过程的发生和发展过程。」
02	**视角性发问**	「我想知道土地改革过程中『情感动员』是怎么进行的。」
03	**差异性发问**	「我想知道为什么土地改革过程在各地区呈现出这些差异。」

> 第一类较难培养；第二类容易培养，但也许会因为缺乏想象力而难以成为一位优秀学者；第三类不但比较容易培养，而且还有成为优秀学者的潜质。

03 · 差别在哪

过程性发问在收集材料，差异性发问在生产解释

前者问「怎么发生的」，后者先看见差异，再问「为什么不一样」。

AI 也一样——它最擅长回答被问清楚的问题。

把问题问成第三种，AI 的输出才会跟别人的不一样。

我的产品：微信悦读，是被一句话问出来的

01

一开始的问法

「帮我做一个微信读书的笔记导出插件。」AI 会很听话地给你一个导出功能——市面上已经有十几款。

02

换成的问法

「为什么有的人几千条笔记越用越有用，我的上万条笔记再也没被翻出来？」这两类人差在哪？

03

结果

答案指向「重新遇见」，而不是「存档导出」——于是有了每日卡片、灵感漫游、读书人性格版。

同一个模型、同一个我，差别只在这句话怎么问。

PART 03

解决一个真实问题的初心

03

不是先有 AI，是先有问题

「微信阅读」

01

它替谁解决什么

给读书多、笔记多的人：让微信读书里的阅读更轻松，让上万条笔记重新被翻出来。

02

为什么做它

自己的笔记记完就死；市面上的十几款插件，要么不好用，要么开始收费。

03

AI 在里面做了什么

写代码、搭结构、改文案——我出问题，它出活。

永久公益、免费、无广告、开源。

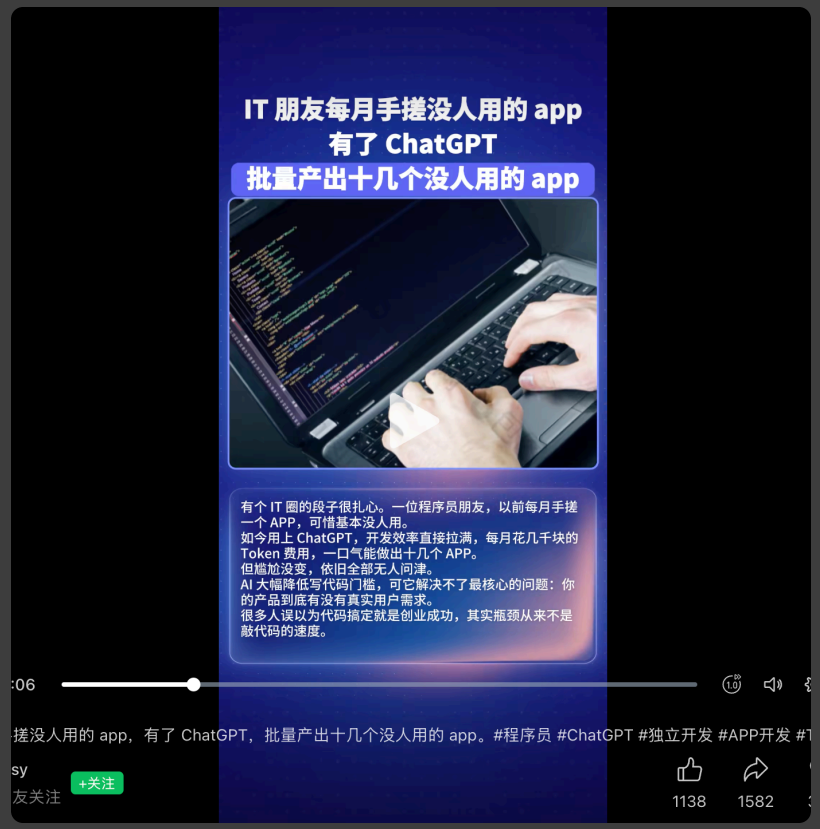
02 · 初心

先有真实的问题，再谈 AI

不解决真问题的创作，做得再快也只是练习。

「我一开始只想做一个小插件。别一上来就想做平台。」

效率拉满了，需求还是零



以前

每月手搓一个 App · 可惜基本没人用。

现在

有了 ChatGPT · 每月几千块 Token，一口气十几个 App。

没变

依旧全部无人问津 · 降下来的是写代码的门槛，不是找需求的门槛。

> 瓶颈从来不是敲代码的速度——AI 让「做出来」变便宜，让「做对」更贵。

PART 04

一套工具

然后才是方法

04

我的真实系统 · 不是示意图

AI 协作架构

一套能复用的工具与经验体系：你出问题，它出活；你把经验存下来，它下次不用再学一遍。

输入端

我

- **真问题**
  差异在哪、为什么
- **判断**
  这样对不对、要不要
- **标准**
  什么算好、什么不许改

→

Trae 工作台 + DeepSeek 大模型

能读 · 能写 · 能跑 · 能开浏览器

规则 RULES
常驻约束，每次对话都生效。**定身份、定红线、定目录。**
例：首末页一个字不动；文件只放 00_素材 / 01_内容结构 / 03_输出。

技能 SKILL
把「做对一次」的方法固化成 SOP + 脚本，**按需加载。**
例：course-ppt、公众号加工、平台分析、取伙发布、转写稿阅读版。

命令 COMMAND
入口。把想做的一件事**说成一句话**，就调起整套流程。
例：「按模板出一版课件」「改成公众号」。

工具 TOOLS
它的手脚：读写文件、跑 Python、操作浏览器、生成 pptx。
例：build_ppt.py 排版分页；浏览器填表到发布按钮前。

→

产出端

课件 PPT
结构文件进，v1/v2 出

文章
纪要 → 公众号版

软件产品
小程序 / 网页工具 / 脚本

记忆底座 OBSIDIAN

`00_素材/ 初稿` `01_内容结构.md` `03_输出/ v1…vN` `版本记录.md` 下次提问直接可读

提问 → 调用 → 产出 → 回写 → 下次更快

三个词，讲明白

规则、技能、命令，到底分别管什么

一层管「不许乱来」，一层管「怎么做对」，一层管「怎么开口」。

概念	是什么	解决什么问题	我的真实例子
规则 RULES	写在项目里的**长期约定**，不用每次重复交代，AI 每次都先读它。	防止跑偏：改错文件、动不该动的地方、输出格式每次都不一样。	首末页原样保留；内容只来自 01_内容结构.md；每次生成新版本、不覆盖旧文件。
技能 SKILL	把一套**已经验证过的做法**写成文字 + 脚本，AI 遇到这类活就照着做。	防止从零开始：每次都要重新解释流程、重新踩坑、质量忽高忽低。	course-ppt：一份结构文件 → 一版课件，自动排版、自动分页、自动记版本。
命令 COMMAND	你开口的**那一句话**（或一个 / 指令），用来精确点名要哪个技能。	防止说不清：需求散在聊天里，AI 猜你的意思，来回改。	「按模板出一版课件」= 调 course-ppt；「改成公众号」= 调公众号加工。

> 没有**规则**，每次都要重新交代底线；没有**技能**，每次都要重新教一遍手艺；没有**命令**，每次都要重新描述一遍想要什么。
> 三者叠起来，才是**「一句话 → 稳定结果」**，而不是「聊半天 → 看运气」。

现场拆解 · 这堂课自己

一份初稿，怎么变成一版课件

每一步都有文件落地，所以随时能回滚、能重来、能换人接手。

01 · 我

写下真实想法

`00_素材/初稿.md`

口语、跳跃、有情绪都没关系，原话保留。

→

02 · 它

拆成页面结构

`01_内容结构.md`

章节 → 页；缺的标【待补】，不替我编。

→

03 · 它

按模板生成

`build_ppt.py`

套版式、分页、检查溢出与缺图。

→

04 · 它

交付新版本

`03_输出/…_v1.pptx`

旧版本不覆盖，就是回滚点。

→

05 · 我

只改一处

再跑一次 → v2

改的是结构文件，不是 PPT 页面。

我负责（不可外包）

- 提一个真实、具体、带差异的问题
- 判断哪句是重点，哪句要停三秒
- 定审美与底线：什么不许改
- 对结果负责

AI 负责（可外包）

- 查资料、读长文、做对照
- 把口语整理成结构
- 套版式、跑脚本、批量重复
- 记版本、留痕、随时重做

这页 PPT，也是这么出来的。

一份课件是怎么生产出来的

把「排版的手艺」变成「可重复的流程」：内容一份、模板一套、脚本一支，改哪里都不会乱。

内容层 我写

00_素材/

初稿 · 录音转写 · 参考文档 · 图片

↓

01_内容结构.md

唯一内容源。原文整段进，不删、不改写；缺的标【待补】。

section · quote · statement · list · cards · timeline

→

版式层 模板 + 脚本

课件模版.pptx

首页与末页原样保留，一个字不动。

↓

build_ppt.py

按页型套版式、自动分页、自动写目录页码、检查缺图与溢出。

深灰 + 金色 · PingFang SC · 16:9

→

版本层 它记

03_输出/ v1 v2 v3 …

每次生成都是新文件，旧版本不覆盖，就是回滚点。

↓

版本记录.md

每次自动追加：版本号、页数、这一次改了什么。

可回滚 · 可对比 · 可接手

迭代公式 改 **01_内容结构.md** → 再跑一次脚本 → 得到 **v(N+1)** | 人不碰页面，只碰内容

内容与版式分离，才换得来「改一句、出一版」

真实案例 · 微信悦读

一个产品是怎么长出来的

起点不是技术，是一句「这件事没那么难，它本来就该是免费公开的」。

01 **先有一个自己的痛点**

上万条笔记再也没被翻出来；十几款插件，要么不好用，要么用着用着开始收费。

02 **收敛成两个命题**

怎么让微信读书里的阅读更轻松？怎么让笔记重新焕发活力？这就是要解决的全部。

03 **先找地基**

微信读书开放了官方 API，能拿到自己的全部阅读数据——先有数据，才谈功能。

04 **只做最小可用版**

「我一开始只想做一个小插件。别一上来就想做平台。」

05 **边做边长**

插件 → 小程序（悦读且住）→ H5 → 网站，四件套才齐。

长出来的三个功能

每日卡片 让死去的知识，重新和你建立关联

灵感漫游 把零散的笔记，拼成一篇有结构的文章

读书人性格版 从阅读数据里算出 16 种人格

「我要做的是一个产品，而不是一个工具——好的产品，是能帮助到人的，以及让人成长。」

—— 我用 7 天做出一款公益产品

我的原话 「模型只是员工，它不够好，不是你的问题。你有没有当好这个老板，才是问题。」

三件套

Trae

编程工具 · 负责把想法落成能跑的文件。

DeepSeek

大模型 · 负责想、写、改、查。

Obsidian

知识管理 · 负责把素材和结构存住、串起来。

> 工具会换，问题不会。

02 · 最后一页之前

这个 PPT 也是我和 AI 协作出来的

从初稿到版式，每一步都在同一个文件里迭代。

所以今天讲的不是理论，是我正在用的做法。

四样东西带走

01	**《这就是ChatGPT》**	AI 到底怎么工作，一本书讲清楚。
02	**《深度学习革命》**	凯德·梅茨写的深度学习史：辛顿、杨立昆、本吉奥怎样熬过两次寒冬，把神经网络送上牌桌。
03	**花叔**	独立开发者，代表作小猫补光灯（App Store 付费榜 Top1）；著有《一本书玩转 DeepSeek》、《AI 编程橙皮书》系列。
04	**质性社会学那篇论文**	学怎么问出一个真正的问题。

> 提问的能力，比工具活得更久。

END

最后，祝你成为超人。

在我的语境中有两层意思

在生活中游刃有余的人

「人应该是被超越的某种东西。」—— 尼采

Chester